


```
EZE Test mit Playwright
test("user can log in", async ({ page }) => {
  await page.goto("/");
  // getbyLabel nutzt sichtbare Labels und fördert dadurch zugängliche Forms.
  await page.getByLabel("Passwort").fill("secret");
  // getbyRole = semantischer und robuster als CSS oder generische Selektoren.
  await page.getByRole("button", { name: "Login" }).click();
  await expect(page.getByRole("status")).toHaveText("Login erfolgreich");
});

Unit + API Test mit Vitest und request von Supertest
describe("calc", () => { test("calculates ...", () => {
  const getTimeoutHour = () => 10; // Arrange
  expect(calc(1000, getTimeoutHour)).toBe(10); // Act and Assert
  expect(() => calc(0, false)).toThrow();
}); });

describe("accounts API", () => {
  test("GET /accounts/1 returns an account", async () => {
    const res = await request(app).get("/accounts/1");
    // request(app).post("/accounts/1/deposit").send({ amount: -50 });
    expect(res.status).toBe(200);
    expect(res.body).toMatchObject({ id: 1, owner: "Ada" }); // Object{error: ""}
    expect(typeof res.body.balance).toBe("number");
  });
});
```

SECURITY

XSS Schutz

- 1. **Output Escaping**: ``${input}``, `{input}`, ``${input}`` // hbs, ``${input}`` // hbs
 - 2. **Sanitizing** (Zeichen entfernen) **beim Speichern** `res(input)`
 - 3. **Content Security Policy** (CSP): blockiert Script-Ausführung im Browser + Server sendet Info im Seiten-Header. Z.B. mit «Helmet» app.use(helmet.contentSecurityPolicy(...)). CSP = wohin der Browser (Fetch)-Requests senden darf.
 - 4. **HttpOnlyCookies**: Statt JWT im localStorage / sessionStorage
- ```
res.cookie('session', token, {
 httpOnly: true, // nicht via document.cookie lesbar
 secure: true, // nur über HTTPS
 sameSite: 'lax' // CSRF-Schutz
});
```

**Angriffs-Szenario**  
Setze user-input auf: `<script>document.body.insertAdjacentHTML('afterbegin', 'cp style="color:red">HACKED</p></script>`

CORS

**CORS** (Cross Origin Resource Sharing) : Browser entscheidet ob Response für JS zugänglich ist, nutzt *Access-Control-Allow-Origin*.

Nicht verwechseln mit CSP: Content Security Policy (CSP) begrenzt, was der Browser laden und ausführen darf → reduziert Schaden bei XSS (aber ersetzt kein Escaping)  
**SOP**: Same-Origin-Policy erlaubt XMLHttpRequest nur zur Origin.  
**Cross-Origin Resource Sharing**: Mechanismus um Cross-Site-Requests zu ermöglichen, Der Ziel-Server kann dem Client den Zugriff erlauben, Wird vom Browser Enforced

**Angriffs-Szenario**  
1. User nutzt app.example.com mit Daten von api.example.com.  
2. Nutzer öffnet aus Versehen evil.com im Browser  
3. evil.com Seite versucht request: `fetch("https://api.example.com/user", {credentials: "include"})` // Cookies mitgesendet

CSRF CROSS-SITE REQUEST FORGERY

**Angriffs-Szenario**  
1. Nutzer ist eingeloggt (Session aktiv)  
2. Nutzer besucht eine Fremde Seite (Attacker)  
3. Diese Seite sendet (Form-Submit oder fetch) eine Anfrage an unsere App  
4. Browser sendet automatisch das Session-Cookie mit  
5. Server führt die Aktion aus (denkt: legitimer Nutzer)

**CSRF Schutz**  
**CSRF Token** Server prüft: kommt Request aus meiner App? und **Keine State Changes** via GET → besonders unsicher

```
// Formular (Client)
<form method="POST" action="/change-email">
 <input name="email" />
 <input type="hidden" name="csrfToken" value="{token}" />
 <button>Save</button>
</form>

// Server (Express Form-Handling)
app.post('/change-email', (req, res) => {
 if (req.body.csrfToken === session.csrfToken) {
 return res.status(403).send("Forbidden");
 }
 email = req.body.email;
 res.send("OK");
});
```

**Cookies**  
ermöglichen Authentication nicht Authorisation Wichtige Cookie Optionen  
– httpOnly → Cookie nicht via JS zugreifbar  
– secure → nur über https gesendet  
– sameSite: lax (default, gesendet bei same-site + top-level Navigation/GET); None (immer), [strict]  
– maxAge / expires: Lebensdauer

LOGIN UND PASSWORT MANAGEMENT

- UX: Re-Authentication (Passwort bestätigen) und Bestätigungs-Dialog vor irreversiblen Aktionen für security and gates design
- Login selber bauen fast immer schlechte Idee, nutze existierende Dienste (Auth0, Firebase Auth...) oder Passwortlos (Magic Link, Passkey).
- schlechte UX = Passwort-Reuse, Fehler → Frust → Workarounds.

**Sign-up Form Best Practices**  
**Nutze den Browser**: richtige HTML-Elemente `<form>`, `<label>`, `<button>`, `<type>`, required, autocomplete nutzen  
**Reduziere «Friction»**: keine doppelte Eingabe (Email / Passwort), klare Fehlermeldungen, Passwortregeln klar anzeigen  
**Passwort UX richtig machen**: Browser Password Manager nutzen, autocomplete="new-password"<current-password>, Show password anbieten  
**Mobile First & Accessibility**: Grosse Inputs und Buttons, Jedes Feld mit <label>

IDOR

**Angriffs-Szenario**  
Der Angreifer loggt sich ein und manipuliert die URL der StockPortfolio Seite von `www.retireEasy.com/stockportfolio/attackerUID` zu `www.retireEasy.com/stockportfolio/victimUID`. Diese Seite wird angezeigt.

```
IDOR Schutz
app.get('/api/documents/:id', async (req, res) => {
 const doc = await db.getDocument(req.db, req.params.id);
 if (!doc || doc.ownerId !== req.session.userId) {
 return res.status(404).json({}); // Enumeration verhindern
 }
 res.json(doc);
});
```

Object-level Authorization-Check: Gehört das Objekt dem aktuellen User?

OWASP TOP 10

A01-2025: Broken Access Control (IDOR, CSRF) A03-2025: Software Supply Chain Failures (Vulnerabel & Outdated Components → NPM) A05-2025: Injection: Cross-Site-Scripting (XSS), JS Code Injection, SQL Injection A06-2025: Insecure Design (→ Meine Impfungen: Nicht gesetzeskonforme und überwindbare Legitimationsprüfung (Brute-Force + IDOR + CSRF + XSS))

| OWASP Top 10:                     |                                         |
|-----------------------------------|-----------------------------------------|
| Cross-Site-Scripting              | Replay Attack                           |
| Remote Code Execution             | Cryptographic Failures                  |
| Insecure Direct Object References | Identification & Authentication Failure |
| Cross-Site Request Forgery        |                                         |

**Cross Site Scripting (XSS)**: Website besitzt eine XSS-Verwundbarkeit, wenn es möglich ist den Server so zu manipulieren, dass Schadcode (JavaScript) eines Angreifers an Nutzer ausgeliefert wird und im Browser dieser Nutzer ausgeführt wird. Gegenmassnahme mit Input Sanitation.

**Remote Code Execution**: Webserver besitzt eine Code Injection "Vulnerability" wenn ein Angreifer den Server dazu bringen kann vom Angreifer eingeschleusten Code zum Ausführen zu bringen.

**Broken Access Control**: Beim behandeln von Formular-Submission-Requests (GET und POST) sollte überprüft werden, dass dies von einem vom Server für den Nutzer «frisch» ausgelieferte Formulare stammt.

**Cryptographic Failures**: Passwort oder Token wird nicht verschlüsselt übertragen. Unabhängig davon ob Query-Body/Request-Parameter.

**Identification & Authentication Failure**: Bei Problemen bei der Authentisierung und dem Session Management können externe Angreifer oder Angreifer mit einem validen Login auf Informationen zugreifen, welche nicht für sie bestimmt sind

- Sign-up Form Best Practices**:
- Use meaningful HTML elements: form, input, label and button
  - Label each input with a label
  - Use element attributes to access built-in browser features: type, name, autocomplete, required.
  - Use autocomplete="new-password" and id="new-password" for the password input in a sign-up form, and for the new password in a reset-password form.
  - Provide Show password functionality.
  - Don't double-up inputs. Don't force users to enter emails or passwords twice.

**Content Security Policy (CSP)**: CSPs ermöglichen die Ausführung von bösamtem Code auf Webseiten zu verhindern. Quellen von Ressourcen (Skripten, Bilder etc.) können eingeschränkt werden. CSPs werden im HTTP Header definiert

**Cross-Origin Resource Sharing (CORS)**: CORS Header ermöglichen Ressourcen (wie z.B. Bilder, Skripte oder Daten) von einem anderen Ursprungsort als der eigenen "Origin" zu laden.